Universidad Nacional

Sede Regional Brunca – Campus Coto

BlaxtorGym

Plan de desarrollo modular

Fases, dependencias, archivos a tocar y checklist de progreso

Integrantes: Justin Rojas Jarquin
Cristopher Ureña Valverde

Tecnologías: C#, WPF, .NET 10, SQLite, Entity Framework Core

Estrategia: primero dominio, persistencia, reglas de negocio e integración. El diseño visual definitivo se realiza cuando el sistema sea funcional.

Documento de seguimiento técnico del proyecto

Índice

1. Cómo utilizar este documento **2**
1.1. Convención actual del proyecto 2
1.2. Punto actual del desarrollo 2

2. Mapa general de fases **2**

3. Especificación detallada por fase **3**

4. Ruta crítica y paralelismo **14**

5. Principios técnicos obligatorios **14**

6. Definition of Done general **15**

1. Cómo utilizar este documento

Este plan funciona simultáneamente como guía de implementación y lista de control. Cada fase contiene su responsable, dependencias, archivos a crear o modificar, tareas concretas y criterio de terminado.

Convención de checklist:

- ☐ Marcar la casilla de la fase cuando toda la fase esté terminada.
- ☐ Marcar individualmente cada archivo cuando haya sido creado o modificado y compile correctamente.
- ☐ Marcar cada tarea solamente cuando su comportamiento haya sido comprobado.
- ☐ No marcar una fase solo porque la pantalla abre.

El número de fase representa una dependencia lógica y no una fila de espera. Justin y Cristopher pueden trabajar en paralelo cuando las dependencias lo permitan.

1.1. Convención actual del proyecto

BlaxtorGym es una sola aplicación WPF. Las capas se mantienen como carpetas lógicas dentro del mismo proyecto. Se conserva el nombre físico `BlaxtorGym.Infraestructure` mientras se desarrolla, para evitar cambios de namespace innecesarios a mitad de implementación.

1.2. Punto actual del desarrollo

Actualmente la Fase 1 ya fue realizada y la Fase 2 está en ejecución. La aplicación ya comprobó la cadena:

WPF → AppDbContext → EF Core → SQLite → blaxtorgym.db

Las casillas se dejan vacías para que el equipo las utilice manualmente.

2. Mapa general de fases

Check	Responsable	Fase / módulo	Dependencias
☐	Justin	1. Configuración inicial	Ninguna
☐	Justin	2. SQLite + EF Core	Fase 1
☐	Justin	3. Autenticación / Login	Fase 2
☐	Justin	4. Tipos de cliente	Fase 2
☐	Cristopher	5. Clientes	Fases 2 y 4
☐	Justin	6. Tipos de membresía + Tarifas	Fases 2 y 4
☐	Cristopher	7. Mediciones físicas	Fase 5
☐	Justin	8. Membresías	Fases 5 y 6
☐	Justin	9. Pagos	Fase 8
☐	Cristopher	10. Notificaciones	Fases 5 y 8
☐	Cristopher	11. Backups	Fase 2 estable
☐	Cristopher	12. Reportes / PDF	Fases 5, 7, 8 y 9

Check	Responsable	Fase / módulo	Dependencias
☐	Ambos	13. Bitácora de cambios	Módulos principales
☐	Ambos	14. Integración general	Fases 1–13
☐	Ambos	15. Diseño visual definitivo	Fase 14
☐	Ambos	16. Pruebas y optimización final	Fases 1–15

3. Especificación detallada por fase

☐ Fase 1 — Configuración inicial del proyecto

Responsable: Justin
Dependencias: Ninguna

Objetivo. Dejar una única aplicación WPF compilable, con paquetes, carpetas lógicas y convenciones listas para desarrollar.

Archivos y estructura

- ☐ `BlaxtorGym.csproj`
- ☐ `App.xaml`
- ☐ `App.xaml.cs`
- ☐ `MainWindow.xaml`
- ☐ `MainWindow.xaml.cs`
- ☐ `src/BlaxtorGym.Application/DTOs/`
- ☐ `src/BlaxtorGym.Application/Services/`
- ☐ `src/BlaxtorGym.Domain/Common/`
- ☐ `src/BlaxtorGym.Domain/Entities/`
- ☐ `src/BlaxtorGym.Domain/Enums/`
- ☐ `src/BlaxtorGym.Domain/Interfaces/`
- ☐ `src/BlaxtorGym.Domain/ValueObjects/`
- ☐ `src/BlaxtorGym.Infraestructure/Data/`
- ☐ `src/BlaxtorGym.Infraestructure/Configuration/`
- ☐ `src/BlaxtorGym.Infraestructure/Repositories/`
- ☐ `src/BlaxtorGym.Infraestructure/Services/`
- ☐ `src/BlaxtorGym.WPF/Views/`
- ☐ `src/BlaxtorGym.WPF/ViewModels/`
- ☐ `src/BlaxtorGym.WPF/Resources/`

Tareas

- ☐ Confirmar .NET 10 y WPF.
- ☐ Instalar los paquetes NuGet requeridos.
- ☐ Crear la estructura lógica sin dividir la solución en múltiples proyectos.
- ☐ Mantener XAML temporal válido para que la solución compile durante el desarrollo.

☐ **Criterio de terminado:** la solución compila en limpio y la estructura está lista para implementar módulos.

☐ Fase 2 — Persistencia: SQLite + Entity Framework Core

Responsable: Justin
Dependencias: Fase 1

Objetivo. Construir el modelo persistente completo y comprobar que WPF puede crear y utilizar `blaxtorgym.db`.

Archivos del dominio

- ☐ `src/BlaxtorGym.Domain/Common/BaseEntity.cs`
- ☐ `src/BlaxtorGym.Domain/Enums/RolUsuario.cs`
- ☐ `src/BlaxtorGym.Domain/Entities/Usuario.cs`
- ☐ `src/BlaxtorGym.Domain/Entities/TipoCliente.cs`
- ☐ `src/BlaxtorGym.Domain/Entities/Cliente.cs`
- ☐ `src/BlaxtorGym.Domain/Entities/TipoMembresia.cs`
- ☐ `src/BlaxtorGym.Domain/Entities/Precio.cs`
- ☐ `src/BlaxtorGym.Domain/Entities/Membresia.cs`
- ☐ `src/BlaxtorGym.Domain/Entities/Pago.cs`
- ☐ `src/BlaxtorGym.Domain/Entities/MedicionFisica.cs`
- ☐ `src/BlaxtorGym.Domain/Entities/Notificacion.cs`
- ☐ `src/BlaxtorGym.Domain/Entities/BitacoraCambio.cs`

Archivos de persistencia

- ☐ `src/BlaxtorGym.Infraestructure/Data/AppDbContext.cs`
- ☐ `src/BlaxtorGym.Infraestructure/Data/DatabaseInitializer.cs`
- ☐ `src/BlaxtorGym.Infraestructure/Configuration/DatabaseSettings.cs`
- ☐ `App.xaml.cs`
- ☐ `Migrations/`

Tareas

- ☐ Definir entidades y relaciones sin implementar todavía los casos de uso de cada módulo.
- ☐ Configurar tablas, campos requeridos, longitudes, índices y relaciones en AppDbContext.
- ☐ Configurar la ruta y cadena de conexión SQLite.
- ☐ Crear automáticamente la carpeta Data de ejecución.
- ☐ Comprobar la creación física de blaxtorgym.db.
- ☐ Completar el modelo antes de fijar el esquema definitivo.
- ☐ Crear y aplicar migraciones controladas.
- ☐ Agregar únicamente los datos iniciales necesarios mediante seeding.
- ☐ Comprobar relaciones e índices en SQLite.

☐ **Criterio de terminado:** `blaxtorgym.db` se crea correctamente; todas las tablas, relaciones e índices esperados existen; el seeding funciona y la aplicación inicia sin errores.

Punto de continuación actual

El siguiente archivo a implementar es:

`src/BlaxtorGym.Domain/Entities/Precio.cs`

☐ Fase 3 — Autenticación / Login

Responsable:	Justin
Dependencias:	Fase 2 completa

Objetivo. Implementar HU-01 con BCrypt, usuario activo/inactivo y bloqueo temporal después de intentos fallidos.

Archivos

- ☐ `src/BlaxtorGym.Domain/Interfaces/IUsuarioRepository.cs`
- ☐ `src/BlaxtorGym.Infraestructure/Repositories/UsuarioRepository.cs`
- ☐ `src/BlaxtorGym.Application/DTOs/LoginRequestDto.cs`
- ☐ `src/BlaxtorGym.Application/Services/AutenticacionService.cs`
- ☐ `src/BlaxtorGym.WPF/ViewModels/LoginViewModel.cs`
- ☐ `src/BlaxtorGym.WPF/Views/LoginWindow.xaml`
- ☐ `src/BlaxtorGym.WPF/Views/LoginWindow.xaml.cs`
- ☐ `App.xaml.cs`

Tareas

- ☐ Buscar el usuario de forma eficiente.
- ☐ Verificar la contraseña con BCrypt.
- ☐ Rechazar usuarios inexistentes.
- ☐ Rechazar usuarios inactivos.
- ☐ Incrementar intentos fallidos por contraseña incorrecta.
- ☐ Bloquear temporalmente después de tres intentos consecutivos fallidos.
- ☐ Restablecer los intentos después de un inicio correcto.
- ☐ Mantener la lógica fuera del code-behind.

☐ **Criterio de terminado:** los cinco escenarios de HU-01 funcionan y tienen sus pruebas principales.

☐ Fase 4 — Tipos de cliente

Responsable: Justin
Dependencias: Fase 2

Objetivo. Administrar el catálogo de tipos de cliente que posteriormente participará en tarifas.

Archivos

- ☐ `src/BlaxtorGym.Domain/Interfaces/ITipoClienteRepository.cs`
- ☐ `src/BlaxtorGym.Infraestructure/Repositories/TipoClienteRepository.cs`
- ☐ `src/BlaxtorGym.Application/Services/TipoClienteService.cs`
- ☐ `src/BlaxtorGym.WPF/ViewModels/TiposClienteViewModel.cs`
- ☐ `src/BlaxtorGym.WPF/Views/TiposClienteView.xaml`

Tareas

- ☐ Listar tipos activos e inactivos.
- ☐ Crear y modificar tipos de cliente.
- ☐ Evitar duplicados según las reglas definidas.
- ☐ Desactivar sin romper historial.
- ☐ Usar AsNoTracking en consultas de solo lectura cuando corresponda.

☐ **Criterio de terminado:** el módulo funciona con persistencia y desbloquea el desarrollo de Clientes.

☐ Fase 5 — Clientes

Responsable: Cristopher
Dependencias: Fases 2 y 4

Objetivo. Implementar registro, consulta, modificación y desactivación de clientes.

Archivos

- ☐ `src/BlaxtorGym.Domain/Interfaces/IClienteRepository.cs`
- ☐ `src/BlaxtorGym.Infraestructure/Repositories/ClienteRepository.cs`
- ☐ `src/BlaxtorGym.Application/DTOs/Cliente*.cs`
- ☐ `src/BlaxtorGym.Application/Services/ClienteService.cs`
- ☐ `src/BlaxtorGym.WPF/ViewModels/ClientesViewModel.cs`
- ☐ `src/BlaxtorGym.WPF/Views/ClientesView.xaml`

Tareas

- ☐ Validar datos requeridos, identificación y contacto.
- ☐ Asociar correctamente el tipo de cliente.
- ☐ Buscar y filtrar desde SQLite en vez de cargar toda la tabla.
- ☐ Modificar sin perder relaciones históricas.
- ☐ Desactivar clientes en vez de eliminarlos.
- ☐ Preparar paginación o carga incremental para listados grandes.

☐ **Criterio de terminado:** HU-02, HU-03, HU-04 y HU-05 funcionan con persistencia y validaciones.

☐ Fase 6 — Tipos de membresía + Tarifas

Responsable: Justin
Dependencias: Fases 2 y 4; puede desarrollarse en paralelo con Clientes

Objetivo. Gestionar duración de membresías y precios por combinación de tipo de cliente y tipo de membresía.

Archivos

- ☐ `src/BlaxtorGym.Domain/Interfaces/ITipoMembresiaRepository.cs`
- ☐ `src/BlaxtorGym.Domain/Interfaces/IPrecioRepository.cs`
- ☐ `src/BlaxtorGym.Infraestructure/Repositories/TipoMembresiaRepository.cs`
- ☐ `src/BlaxtorGym.Infraestructure/Repositories/PrecioRepository.cs`
- ☐ `src/BlaxtorGym.Application/Services/TipoMembresiaService.cs`
- ☐ `src/BlaxtorGym.Application/Services/TarifaService.cs`
- ☐ `src/BlaxtorGym.WPF/ViewModels/TarifasViewModel.cs`
- ☐ `src/BlaxtorGym.WPF/Views/TarifasView.xaml`

Tareas

- ☐ Administrar tipos de membresía y su duración en días.
- ☐ Administrar precios por combinación de tipo de cliente y membresía.
- ☐ Evitar combinaciones inválidas o duplicadas.
- ☐ Conservar historial cuando cambien las tarifas.
- ☐ Consultar eficientemente la tarifa aplicable.

☐ **Criterio de terminado:** las combinaciones configuradas poseen una tarifa válida, vigente y consultable.

☐ Fase 7 — Mediciones físicas

Responsable: Cristopher
Dependencias: Fase 5

Objetivo. Registrar e historizar mediciones físicas asociadas a cada cliente.

Archivos

- ☐ `src/BlaxtorGym.Domain/Interfaces/IMedicionFisicaRepository.cs`
- ☐ `src/BlaxtorGym.Infraestructure/Repositories/MedicionFisicaRepository.cs`
- ☐ `src/BlaxtorGym.Application/Services/MedicionFisicaService.cs`
- ☐ `src/BlaxtorGym.WPF/ViewModels/MedicionesViewModel.cs`
- ☐ `src/BlaxtorGym.WPF/Views/MedicionesView.xaml`

Tareas

- ☐ Registrar fecha y valores definidos por el módulo.
- ☐ Validar rangos y datos requeridos.
- ☐ Mostrar historial ordenado por cliente.
- ☐ Evitar consultas N+1.

☐ **Criterio de terminado:** registro e historial por cliente funcionan y quedan disponibles para reportes.

☐ Fase 8 — Membresías

Responsable: Justin
Dependencias: Fases 5 y 6

Objetivo. Crear y renovar membresías calculando vigencia y estado.

Archivos

- ☐ `src/BlaxtorGym.Domain/Interfaces/IMembresiaRepository.cs`
- ☐ `src/BlaxtorGym.Infraestructure/Repositories/MembresiaRepository.cs`
- ☐ `src/BlaxtorGym.Application/Services/MembresiaService.cs`
- ☐ `src/BlaxtorGym.WPF/ViewModels/MembresiasViewModel.cs`
- ☐ `src/BlaxtorGym.WPF/Views/MembresiasView.xaml`

Tareas

- ☐ Asociar cliente, tipo de membresía y tarifa aplicable.
- ☐ Calcular fecha de inicio y fecha final.
- ☐ Renovar sin destruir historial.
- ☐ Determinar el estado de la membresía de forma consistente.
- ☐ Indexar búsquedas frecuentes por cliente, estado y vencimiento.

☐ **Criterio de terminado:** creación y renovación funcionan con fechas, precios y estados consistentes.

☐ Fase 9 — Pagos

Responsable: Justin
Dependencias: Fase 8

Objetivo. Registrar pagos asociados a membresías y conservar historial financiero.

Archivos

- ☐ `src/BlaxtorGym.Domain/Interfaces/IPagoRepository.cs`
- ☐ `src/BlaxtorGym.Infraestructure/Repositories/PagoRepository.cs`
- ☐ `src/BlaxtorGym.Application/Services/PagoService.cs`
- ☐ `src/BlaxtorGym.WPF/ViewModels/PagosViewModel.cs`
- ☐ `src/BlaxtorGym.WPF/Views/PagosView.xaml`

Tareas

- ☐ Asociar cada pago a una membresía válida.
- ☐ Validar monto y fecha.
- ☐ Consultar historial por cliente, membresía y rango de fechas.
- ☐ Conservar el valor histórico aunque las tarifas cambien después.

☐ **Criterio de terminado:** los pagos quedan persistidos, relacionados y consultables correctamente.

☐ Fase 10 — Notificaciones

Responsable: Cristopher
Dependencias: Fases 5 y 8

Objetivo. Generar avisos de vencimiento y controlar su estado de envío.

Archivos

- ☐ `src/BlaxtorGym.Domain/Interfaces/INotificacionRepository.cs`
- ☐ `src/BlaxtorGym.Infraestructure/Repositories/NotificacionRepository.cs`
- ☐ `src/BlaxtorGym.Application/Services/NotificacionService.cs`
- ☐ `src/BlaxtorGym.Infraestructure/Services/CorreoService.cs`
- ☐ `src/BlaxtorGym.WPF/ViewModels/NotificacionesViewModel.cs`
- ☐ `src/BlaxtorGym.WPF/Views/NotificacionesView.xaml`

Tareas

- ☐ Detectar vencimientos mediante consultas filtradas.
- ☐ Evitar notificaciones duplicadas.
- ☐ Registrar estado pendiente/enviado.
- ☐ No bloquear la interfaz durante correo u otras operaciones de I/O.

☐ **Criterio de terminado:** los avisos se generan y su estado queda registrado y trazable.

☐ Fase 11 — Backups

Responsable: Cristopher
Dependencias: Fase 2 estable

Objetivo. Respaldar y restaurar SQLite de forma controlada.

Archivos

- ☐ `src/BlaxtorGym.Application/Services/BackupService.cs`
- ☐ `src/BlaxtorGym.Infraestructure/Services/BackupService.cs`
- ☐ `src/BlaxtorGym.WPF/ViewModels/ConfiguracionViewModel.cs`
- ☐ `src/BlaxtorGym.WPF/Views/ConfiguracionView.xaml`

Tareas

- ☐ Definir ubicación y nombre de respaldos.
- ☐ Respaldar SQLite de manera segura.
- ☐ Validar un respaldo antes de restaurarlo.
- ☐ Evitar congelar WPF durante copia o restauración.
- ☐ Manejar y registrar errores.

☐ **Criterio de terminado:** backup y restauración han sido probados sin pérdida ni corrupción de datos.

☐ Fase 12 — Reportes / PDF

Responsable: Cristopher
Dependencias: Fases 5, 7, 8 y 9

Objetivo. Construir reportes con datos reales y exportarlos a PDF.

Archivos

- ☐ `src/BlaxtorGym.Application/DTOs/Reporte*.cs`
- ☐ `src/BlaxtorGym.Application/Services/ReporteService.cs`
- ☐ `src/BlaxtorGym.Infraestructure/Services/PdfService.cs`
- ☐ `src/BlaxtorGym.WPF/ViewModels/ReportesViewModel.cs`
- ☐ `src/BlaxtorGym.WPF/Views/ReportesView.xaml`

Tareas

- ☐ Filtrar por rangos desde la base de datos.
- ☐ Proyectar solamente columnas necesarias.
- ☐ Evitar N+1.
- ☐ Separar consulta de datos de la generación del PDF.
- ☐ No congelar la interfaz durante generación costosa.

☐ **Criterio de terminado:** los reportes coinciden con los datos persistidos y los PDF se generan correctamente.

☐ Fase 13 — Bitácora de cambios

Responsable: Ambos
Dependencias: Módulos principales funcionales

Objetivo. Registrar acciones y cambios relevantes dentro del sistema.

Archivos

- ☐ `src/BlaxtorGym.Domain/Interfaces/IBitacoraRepository.cs`
- ☐ `src/BlaxtorGym.Infraestructure/Repositories/BitacoraRepository.cs`
- ☐ `src/BlaxtorGym.Application/Services/BitacoraService.cs`
- ☐ `src/BlaxtorGym.WPF/ViewModels/BitacoraViewModel.cs`
- ☐ `src/BlaxtorGym.WPF/Views/BitacoraView.xaml`

Tareas

- ☐ Definir qué acciones requieren auditoría.
- ☐ Registrar usuario, fecha, acción y referencia necesaria.
- ☐ Evitar que el registro de auditoría bloquee operaciones normales.
- ☐ Paginar el historial cuando crezca.

☐ **Criterio de terminado:** las acciones críticas quedan registradas y pueden consultarse.

☐ Fase 14 — Integración general

Responsable: Ambos
Dependencias: Fases 1–13

Objetivo. Integrar navegación, dependencias, permisos y manejo uniforme de errores.

Archivos

- ☐ `App.xaml`
- ☐ `App.xaml.cs`
- ☐ `MainWindow.xaml`
- ☐ `MainWindow.xaml.cs`
- ☐ `src/BlaxtorGym.WPF/NavigationService.cs`
- ☐ `ViewModels y Services que requieran ajustes de integraciÃşn`

Tareas

- ☐ Eliminar acoplamientos temporales.
- ☐ Centralizar la resolución de dependencias.
- ☐ Validar navegación completa.
- ☐ Aplicar permisos según rol cuando corresponda.
- ☐ Unificar mensajes y manejo de excepciones.

☐ **Criterio de terminado:** el flujo principal funciona de extremo a extremo sin dependencias manuales frágiles.

☐ Fase 15 — Diseño visual definitivo

Responsable: Ambos
Dependencias: Fase 14

Objetivo. Aplicar identidad visual y mejorar UX sin modificar las reglas de negocio.

Archivos

- ☐ `src/BlaxtorGym.WPF/Resources/*.xaml`
- ☐ `src/BlaxtorGym.WPF/Views/*.xaml`
- ☐ `MainWindow.xaml`
- ☐ `Recursos grÃąficos e iconos`

Tareas

- ☐ Definir colores, tipografía y componentes reutilizables.
- ☐ Aplicar estilos centralizados.
- ☐ Diseñar estados vacíos, carga y errores.
- ☐ Virtualizar DataGrid y ListView con muchas filas.
- ☐ Mantener lógica de negocio fuera de code-behind.

☐ **Criterio de terminado:** la interfaz es consistente, usable y responsiva sin regresiones funcionales.

☐ Fase 16 — Pruebas y optimización final

Responsable: Ambos
Dependencias: Todas las fases anteriores

Objetivo. Validar requisitos, regresiones y comportamiento con crecimiento de datos.

Archivos y evidencias

- ☐ `Casos de prueba funcionales`
- ☐ `Proyecto o carpeta de pruebas automatizadas, si se incorpora`
- ☐ `ConfiguraciÃşn y logs de mediciÃşn`
- ☐ `Repositorios, consultas o ViewModels optimizados a partir de mediciones`

Tareas

- ☐ Probar criterios de aceptación de cada módulo.
- ☐ Probar datos inválidos y escenarios de error.
- ☐ Probar regresiones entre módulos.
- ☐ Medir consultas y tiempos con volúmenes crecientes.
- ☐ Revisar índices, AsNoTracking, paginación y N+1.
- ☐ Comprobar que I/O no bloquee WPF.
- ☐ Optimizar únicamente cuellos de botella medidos.

☐ **Criterio de terminado:** el sistema es estable, las pruebas principales pasan y el rendimiento ha sido validado.

4. Ruta crítica y paralelismo

Configuración → Persistencia → Tipos de cliente → Clientes + Tarifas → Membresías → Pagos → Integración

Backups puede adelantarse cuando la base de datos sea estable. Clientes y Tarifas pueden desarrollarse en paralelo cuando sus dependencias estén disponibles. Reportes debe esperar datos reales de Clientes, Mediciones, Membresías y Pagos.

5. Principios técnicos obligatorios

Principio	Aplicación en BlaxtorGym
SOLID pragmático	Una clase tiene una responsabilidad clara. Se usan interfaces donde exista una dependencia real, sin crear abstracciones vacías solo por patrón.
MVVM	La View presenta; el ViewModel maneja estado y comandos; Application Service ejecuta casos de uso; Repository persiste.
EF Core / SQLite	Filtrar, ordenar y proyectar en SQLite; usar AsNoTracking en lectura; evitar N+1; crear índices útiles.
Async/Await	No utilizar `.Result` ni `.Wait()` en el hilo WPF. Las operaciones de I/O no deben congelar la interfaz.
Historial	No destruir información histórica necesaria. Clientes se desactivan; tarifas, membresías y pagos mantienen consistencia histórica.
Rendimiento	Paginar listados grandes, virtualizar controles y evitar colecciones enormes innecesarias.
Errores	No usar catch vacíos. Mostrar mensajes entendibles y conservar detalle técnico para diagnóstico.
Seguridad	Contraseñas con BCrypt y secretos fuera del repositorio.

6. Definition of Done general

Antes de marcar cualquier fase como completada, comprobar:

- ☐ El código compila sin errores.
- ☐ La funcionalidad principal fue probada.
- ☐ Los datos se persisten correctamente cuando corresponde.
- ☐ Las entradas inválidas son rechazadas de forma controlada.
- ☐ Los errores no cierran inesperadamente la aplicación.
- ☐ La interfaz no se bloquea durante operaciones costosas.
- ☐ La fase respeta la separación View – ViewModel – Service – Repository.
- ☐ No se rompieron módulos previamente funcionales.
- ☐ Las pruebas principales del módulo fueron ejecutadas.